

Input e output

File

Come leggere e scrivere file di testo in modo semplice.

Perche usare i file

Finora i dati vivevano solo mentre il programma era aperto. Quando il programma finiva, le variabili sparivano.

Un file permette di salvare informazioni su disco: una lista, un messaggio, un risultato, una configurazione.

In questa pagina usiamo file di testo semplici.

Scrivere un file

Per scrivere testo in un file, puoi usare `FileWriter` .

```
public class ScriviFile {  
    public static void main(String[] args) {  
        try {  
            FileWriter writer = new FileWriter("messaggio.txt");  
            writer.write("Ciao dal file!");  
            writer.close();  
  
            System.out.println("File scritto.");  
        } catch (IOException e) {  
            System.out.println("Errore durante la scrittura.");  
        }  
    }  
}
```

Dopo l'esecuzione, nella stessa cartella del programma troverai `messaggio.txt` .

Perche c'e try catch

Scrivere su un file puo fallire: la cartella potrebbe non esistere, il file potrebbe essere bloccato, oppure potresti non avere i permessi.

Java ti chiede di gestire questa possibilita.

Per ora leggi `try catch` cosi:

- prova a fare questa operazione
- se qualcosa va storto, esegui il blocco `catch`

Vedremo le eccezioni con piu calma nella sezione sugli errori.

Leggere un file

Per leggere un file riga per riga, puoi usare `Scanner` .

Prima crea un file `messaggio.txt` con questo contenuto:

```
Ciao dal file!
```

Poi scrivi:

```

public class LeggiFile {
    public static void main(String[] args) {
        try {
            File file = new File("messaggio.txt");
            Scanner reader = new Scanner(file);

            while (reader.hasNextLine()) {
                String riga = reader.nextLine();
                System.out.println(riga);
            }

            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File non trovato.");
        }
    }
}

```

Output:

```
Ciao dal file!
```

Percorsi relativi

Nel codice abbiamo scritto:

```
new File("messaggio.txt")
```

Questo è un **percorso relativo**. Java cerca il file nella cartella da cui esegui il programma.

Se il programma dice "File non trovato", controlla:

- il nome del file
- l'estensione `.txt`
- la cartella in cui stai eseguendo `java`

Aggiungere testo a un file

Di solito `FileWriter` sovrascrive il file. Se vuoi aggiungere testo alla fine, usa il secondo parametro `true` :

```
FileWriter writer = new FileWriter("diario.txt", true);
writer.write("Nuova riga\n");
writer.close();
```

`\n` indica un a capo.

Regola pratica

Quando lavori con i file, ricordati tre cose:

1. le operazioni possono fallire, quindi servono `try catch`
2. le risorse aperte vanno chiuse
3. il percorso del file dipende dalla cartella in cui esegui il programma

I file sono il primo passo per far comunicare Java con dati esterni al codice.

Input da tastiera

Come leggere dati inseriti dall'utente con Scanner.

Leggere quello che scrive l'utente

L'input è ciò che entra nel programma. Nei primi esempi leggeremo dati dalla tastiera.

Per farlo useremo `Scanner`, una classe della libreria standard di Java.

Prima di usarla, devi importarla:

```
import java.util.Scanner;
```

Un primo esempio

```
public class Saluto {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Come ti chiami? ");  
        String nome = scanner.nextLine();  
  
        System.out.println("Ciao, " + nome + "!");  
    }  
}
```

Esempio di esecuzione:

```
Come ti chiami? Luca  
Ciao, Luca!
```

`System.in` rappresenta l'input che arriva dal terminale.

`nextLine()` legge una riga intera di testo.

Leggere un numero intero

Per leggere un `int`, usa `nextInt()` :

```

public class Eta {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Quanti anni hai? ");
        int eta = scanner.nextInt();

        System.out.println("Tra un anno avrai " + (eta + 1) + " anni.");
    }
}

```

Esempio:

```

Quanti anni hai? 20
Tra un anno avrai 21 anni.

```

Leggere un numero decimale

Per un numero con decimali, usa `nextDouble()` :

```

public class Prezzo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Prezzo: ");
        double prezzo = scanner.nextDouble();

        System.out.println("Doppio prezzo: " + (prezzo * 2));
    }
}

```

Nota: in molti ambienti Java si aspetta il punto per i decimali, per esempio `12.50` , non `12,50` .

Il piccolo problema dopo nextInt

Un errore comune nasce quando usi `nextInt()` e subito dopo `nextLine()` .

```
public class Profilo {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Eta: ");  
        int eta = scanner.nextInt();  
  
        System.out.print("Nome: ");  
        String nome = scanner.nextLine();  
  
        System.out.println(nome + ", " + eta);  
    }  
}
```

Il programma potrebbe saltare la richiesta del nome. Succede perché `nextInt()` legge il numero, ma lascia nel buffer il carattere di a capo.

Una soluzione semplice è consumare quella riga prima di leggere il nome:

```
System.out.print("Eta: ");  
int eta = scanner.nextInt();  
scanner.nextLine(); // consuma l'a capo rimasto  
  
System.out.print("Nome: ");  
String nome = scanner.nextLine();
```

Chiudere lo Scanner

Negli esempi piccoli spesso vedrai:

```
scanner.close();
```

Chiudere lo scanner libera la risorsa usata per leggere.

In programmi brevi da terminale puoi metterlo alla fine:

```
Scanner scanner = new Scanner(System.in);

// letture...

scanner.close();
```

Esempio completo

```
public class Ordine {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Prodotto: ");
        String prodotto = scanner.nextLine();

        System.out.print("Quantita: ");
        int quantita = scanner.nextInt();

        System.out.print("Prezzo: ");
        double prezzo = scanner.nextDouble();

        double totale = quantita * prezzo;
        System.out.println("Totale per " + prodotto + ": " + totale + "
euro");

        scanner.close();
    }
}
```

L'input rende il programma meno fisso: invece di usare sempre gli stessi valori, puoi farli scegliere all'utente.

Output

Come stampare testo e valori con System.out.

Mostrare informazioni nel terminale

L'output è ciò che il programma mostra all'esterno. Nei primi esercizi useremo il terminale.

In Java si stampa con `System.out`.

```
System.out.println("Ciao!");
```

Output:

```
Ciao!
```

println

`println` stampa un valore e poi va a capo.

```
public class Output {  
    public static void main(String[] args) {  
        System.out.println("Prima riga");  
        System.out.println("Seconda riga");  
    }  
}
```

Output:

```
Prima riga  
Seconda riga
```

Ogni chiamata a `println` produce una riga nuova.

print

`print` stampa senza andare a capo.

```
System.out.print("Ciao ");  
System.out.print("Luca");  
System.out.print("!");
```

Output:

```
Ciao Luca!
```

La differenza è piccola ma importante:

- `println` stampa e va a capo
- `print` stampa e resta sulla stessa riga

Stampare variabili

Puoi stampare il valore di una variabile:

```
String nome = "Sara";  
int eta = 21;  
  
System.out.println(nome);  
System.out.println(eta);
```

Output:

```
Sara  
21
```

Concatenare testo e valori

Per costruire un messaggio, usa `+` .

```
String nome = "Sara";  
int eta = 21;  
  
System.out.println("Nome: " + nome);  
System.out.println("Eta: " + eta);
```

Output:

```
Nome: Sara  
Eta: 21
```

Quando uno dei pezzi è una stringa, Java trasforma anche gli altri valori in testo.

Attenzione ai calcoli dentro le stringhe

Guarda questo esempio:

```
System.out.println("Totale: " + 10 + 5);
```

Output:

```
Totale: 105
```

Java parte da sinistra. Appena trova una stringa, unisce tutto come testo.

Se vuoi fare prima il calcolo, usa le parentesi:

```
System.out.println("Totale: " + (10 + 5));
```

Output:

```
Totale: 15
```

Formattare valori semplici

Per stampare un numero decimale con due cifre, puoi usare `printf`.

```
double prezzo = 12.5;
System.out.printf("Prezzo: %.2f euro%n", prezzo);
```

Output:

```
Prezzo: 12.50 euro
```

Qui `%.2f` significa: "stampa un numero decimale con due cifre dopo il punto".

`%n` manda a capo.

Esempio completo

```
public class Scontrino {
    public static void main(String[] args) {
        String prodotto = "Quaderno";
        int quantita = 3;
        double prezzo = 2.50;
        double totale = quantita * prezzo;

        System.out.println("Prodotto: " + prodotto);
        System.out.println("Quantita: " + quantita);
        System.out.printf("Totale: %.2f euro%n", totale);
    }
}
```

Output:

```
Prodotto: Quaderno  
Quantita: 3  
Totale: 7.50 euro
```

L'output e il modo piu semplice per controllare cosa sta facendo un programma.